

Supermarket Checkout System — Project Report

Course: Software Engineering Project 2026 — CentraleSupélec **Language:** Java 17+ (built and tested on JDK 21), Maven, JUnit 5 **Repository:** <https://github.com/enzovendramin/supermarket-checkout>

1. Overview

This project implements a supermarket checkout system in Java with a command-line user interface (CLUI). It processes purchases, computes bills combining customer discount plans and category pricing policies, handles bank card payments through a POS terminal and the bank's Transaction Authorisation System (TAS), manages a live inventory with real-time low-stock alerts, and supports home delivery with weight/distance-based charging and smart logistics (time slots, anti-overbooking, dynamic pricing).

The system is organised in layered packages:

```
supermarket
├─ model      domain entities (Item, Category, Cart, CartEntry, BankCard,
Customer, Receipt)
├─ users      User (abstract) + Manager, Cashier
├─ discount   DiscountPlan strategy + Normal/Prime/Platinum + factory
├─ pricing    PricingPolicy strategy + NoPricingPolicy, CategoryDiscount
├─ inventory  Inventory + StockObserver + Manager/Supplier notifiers (Observer)
├─ payment    POSDevice, TransactionAuthorisationSystem, PaymentSimulator,
PaymentResult
├─ delivery   DeliveryCalculator strategy + DeliveryRequest, TimeSlot,
DeliveryManager
├─ register   CashRegister (checkout orchestration) + Supermarket (system core)
└─ cli        CLI, CommandContext, CommandDispatcher (the CLUI)
```

2. Design Decisions — How Requirements Shaped the Code

Requirement	Design impact
R1, R2, R2b — purchases are sets of bought items in mandatory categories	Cart aggregates CartEntry (item + quantity); Category is a first-class entity so policies can attach to it.
R3 — each item is priced	Item holds a unit price; CartEntry.subtotal() multiplies by quantity.
R4 — payment by bank card	POSDevice + TransactionAuthorisationSystem model the POS↔bank protocol; BankCard carries number, PIN and balance.
R5 / R5b — extensible discount plans	Strategy interface DiscountPlan with a Factory to create plans by name at runtime, so new plans need no changes to the checkout.

Requirement	Design impact
R6 / R6b — extensible per-category pricing policies	Strategy interface PricingPolicy attached to each Category ; new categories/policies plug in freely.
R7, R8, R8b — home delivery with weight/distance charging and plan discounts	DeliveryCalculator Strategy computes the fee; the customer's DiscountPlan reduces/waives it.
R9 — live inventory and low-stock alerts	Observer pattern: Inventory notifies StockObservers when a perishable item drops to/below its threshold.
R10 — 2-hour slots, anti-overbooking, dynamic pricing	TimeSlot + DeliveryManager track capacity per window and compute peak/eco price multipliers.
Part 2 — CLUI with error handling	Three-class CLI layer with a quote-aware tokenizer, role-based permissions, and uniform error reporting that never crashes the interpreter.

3. Design Patterns Used

Pattern	Where	Problem it solves
Strategy	DiscountPlan → Normal/Prime/Platinum	Swap discount behaviour per customer without conditionals; satisfies the <i>extensibility</i> requirement R5b.
Strategy	PricingPolicy → NoPricingPolicy/CategoryDiscount	Per-category pricing that can be changed/added at runtime (R6b).
Strategy	DeliveryCalculator → StandardDeliveryCalculator	The delivery charging scheme can be replaced without touching the bill logic (R8).
Factory	DiscountPlanFactory	Creates a plan from its string name (used by subscribeToPlan), centralising the mapping and isolating the rest of the system from concrete plan classes.
Observer	Inventory → StockObserver (ManagerNotifier , SupplierNotifier)	Decouples stock bookkeeping from who reacts to low stock; new listeners can subscribe without modifying Inventory (R9, explicitly required).
Facade	TransactionAuthorisationSystem	Hides the banking authorisation protocol behind a single authorise(card, amount) method.
(Mediator-like)	CashRegister / Supermarket	Supermarket is the system core wiring components together; CashRegister orchestrates a checkout session across cart, plan, delivery, POS and inventory.

4. Bill Computation

`CashRegister.computeBill()` applies discounts in this order (spec §2.4):

1. **Per-item category policy** (R6): each item price is passed through its `Category.pricingPolicy` (e.g. –10% on fruit-and-vegetables).
2. **Customer discount plan** (R5): the resulting subtotal is passed through the customer's `DiscountPlan` (e.g. Prime –20% only if subtotal $\geq$ €50; Platinum –30% always).
3. **Delivery fee** (R8/R8b): if a delivery was requested, the `DeliveryCalculator` computes the fee from total weight, distance and order value, then the plan's `applyDeliveryDiscount` reduces it (Prime 50%, Platinum free).

`total = items_after_plan + delivery_after_plan.`

5. UML Diagrams

The diagrams are provided as PlantUML sources in `docs/uml/`:

File	Diagram
<code>class-diagram.puml</code>	Class diagram (mandatory) — all packages, the three Strategy hierarchies, the Observer hierarchy, and the orchestration core.
<code>use-case-diagram.puml</code>	Use cases for Manager, Cashier, Customer and the Bank.
<code>sequence-checkout-payment.puml</code>	Checkout + bill + payment flow, including forced/real payment and the post-success inventory decrement.
<code>sequence-low-stock-alert.puml</code>	The Observer notification when a perishable item drops below threshold.

Rendering: open the `.puml` files with the *PlantUML* extension in VS Code, or paste them into <https://www.plantuml.com/plantuml> to export PNG/SVG/PDF for inclusion in the final PDF.

6. Test Scenarios (mandatory description)

Two scenario files are provided and run via `runTest <file>` in the CLUI.

`testScenario1.txt`

End-to-end "happy path" exercising the core requirements:

1. Manager bootstraps the system (`setup`) and extends the catalogue.
2. Category-level pricing policy: –10% on *fruit-and-vegetables* (R6).
3. Customer subscribes to *prime* (R5) and requests home delivery (R7).
4. Cashier opens a checkout with a mixed cart of all three mandatory categories (R2b).
5. `computeBill` combines the category discount and the plan; delivery fee is halved by the prime plan (R8b).
Result: items €23.24 + delivery €7.50 = €30.74. (Prime's 20% does *not* apply here because the subtotal is below €50 — correct per R5.)
6. First payment fails (`INSUFFICIENT_FUNDS`), retry succeeds (R4).

7. A second checkout buys 4 steaks (€50 → prime −20% = €40) and drives steak stock to 0, **firing the R9 low-stock alert** (Observer).
8. Manager inspects `showInventory` (steak flagged [`LOW STOCK`]) and `showRevenue` (**€70.74**).

testScenario2.txt

Complementary scenario covering paths scenario 1 does not:

1. **Platinum** plan: −30% on items and **free delivery** (R5, R8b).
2. Category discount on *dairy* (−5%, R6).
3. Payment unhappy paths forced via `simulatePayment`: **PIN_WRONG** then **AUTH_DENIED**, then a successful payment (R4).
4. A low-stock alert on *beef* (R9) and its **clearing via restock**.
5. **Home-delivery refusal for an order above 50 kg** (R8). Final revenue: **€54.236**.

7. How to Test the Realisation (mandatory)

Build & run the CLUI

```
mvn compile
mvn exec:java -Dexec.mainClass="supermarket.cli.CLI"
```

At startup the system auto-loads `my_supermarket.ini` (the standard setup), then accepts commands. To run a scenario from inside the CLUI:

```
runTest testScenario1.txt
runTest testScenario2.txt
```

Type `help` to list all commands, `exit` to quit.

JUnit tests

```
mvn test
```

54 tests cover the system. They are organised one test class per unit:

Test class	What it verifies
<code>BillComputationTest</code>	Category + plan discount math, prime €50 threshold, platinum.
<code>PaymentFlowTest</code>	Forced + real POS outcomes (success/insufficient/PIN/denied), stock decrement.

Test class	What it verifies
<code>InventoryObserverTest</code>	Observer fires only for perishables at/below threshold; restock.
<code>DeliveryCalculatorTest</code>	Flat fee, flat+percentage, >50 kg refusal.
<code>DeliveryManagerTest</code>	Anti-overbooking and peak/eco dynamic pricing.
<code>DeliveryCheckoutTest</code>	End-to-end prime delivery fee (€7.50) on the scenario cart.
<code>CliDispatcherTest</code>	Both scenarios end-to-end, quote-aware parsing, permission/syntax/misuse errors.
<code>DiscountPlanTest</code> , <code>PricingPolicyTest</code> , <code>CartTest</code> , <code>BankCardTest</code> , <code>TimeSlotTest</code> , <code>PaymentSimulatorTest</code> , <code>TransactionAuthorisationSystemTest</code> , <code>SupermarketTest</code>	Focused unit tests per class.

Reproducing the tests is a single `mvn test`; the scenario files are read from the project root, so the JUnit scenario tests and the CLUI `runTest` execute the exact same command sequences.

8. Design Analysis — Advantages & Limitations

Advantages

- **Extensibility by design:** new discount plans, pricing policies and delivery schemes are added by implementing one interface — no edits to the checkout (Open/Closed Principle). New low-stock listeners just subscribe to `Inventory`.
- **Separation of concerns:** domain model, services (payment/inventory/delivery) and the CLUI are independent layers; the core logic is fully unit-testable without the CLUI (the dispatcher writes to an injectable `PrintStream`).
- **Reproducible, locale-independent output** (`Locale.US`, UTF-8), so the bill figures are identical on any grader's machine.
- **Robust CLUI:** all syntax/misuse errors are reported without crashing.

Limitations / possible improvements

- **Distance is a stub** (`DEFAULT_DISTANCE_KM = 10`): a real system would geocode the address. It is isolated so it can be replaced by a strategy.
- **R10 logistics is a standalone module:** `TimeSlot`/`DeliveryManager` are fully implemented and unit-tested but not yet exposed as dedicated CLUI commands (the 20-command spec does not include slot booking).
- **In-memory persistence:** state lives in memory for the duration of a run; there is no database.
- **Single cash register:** the model assumes one active checkout at a time, which matches the scenarios but could be generalised to several registers.

9. Workload Split (mandatory)

Fill the **Member** column with each team member's name. Columns follow the mandatory format: *design | code | UML | JUnit | task/class*.

Module / Task	Design	Code	UML	JUnit	Member
Domain model (<i>model</i> , <i>users</i>)	✓	✓	✓	<i>CartTest, BankCardTest</i>	<i>TBD</i>
Discount plans (<i>discount</i> , <i>Strategy+Factory</i>)	✓	✓	✓	<i>DiscountPlanTest</i>	<i>TBD</i>
Pricing policies (<i>pricing</i> , <i>Strategy</i>)	✓	✓	✓	<i>PricingPolicyTest</i>	<i>TBD</i>
Checkout & bill (<i>register.CashRegister</i>)	✓	✓	✓	<i>BillComputationTest</i>	<i>TBD</i>
Payment (<i>payment</i> , <i>POS/TAS, Facade</i>)	✓	✓	✓	<i>PaymentFlowTest, TransactionAuthorisationSystemTest, PaymentSimulatorTest</i>	<i>TBD</i>
Inventory & alerts (<i>inventory</i> , <i>Observer</i>)	✓	✓	✓	<i>InventoryObserverTest</i>	<i>TBD</i>
Delivery & logistics (<i>delivery</i> , <i>R8/R10</i>)	✓	✓	✓	<i>DeliveryCalculatorTest, DeliveryManagerTest, TimeSlotTest</i>	<i>TBD</i>
System core (<i>register.Supermarket</i>)	✓	✓	✓	<i>SupermarketTest</i>	<i>TBD</i>
CLUI (<i>cli</i>) + scenarios	✓	✓	✓	<i>CliDispatcherTest, DeliveryCheckoutTest</i>	<i>TBD</i>
Report & UML diagrams	✓	—	✓	—	<i>TBD</i>

10. How to Build the Environment

```
# Requires JDK 17+ (developed on JDK 21) and Maven 3.9+
mvn clean test          # compile + run all 54 JUnit tests
mvn exec:java -Dexec.mainClass="supermarket.cli.CLI" # launch the CLUI
```